

Les événements en JavaFX

I. Petit rappels sur JavaFX

Si vous êtes au point, passez directement à la partie II sur les événements.

Les 3 classes de base d'une application JavaFX : Application, Stage et Scene

Pour démarrer une application JavaFX, on procède toujours de la même manière (de fait, la classe principale d'une application JavaFx ne change pas d'un projet à l'autre) :

- La classe principale d'une application javaFX hérite de la classe `javafx.application.Application`. C'est la méthode `start()` le point d'entrée pour toute application JavaFX.
- Cette classe `javafx.application.Application` gère tout le cycle de vie de l'application pour vous (ouverture des fenêtres, initialisations, le démarrage et la fin de l'application, etc).
- Après avoir exécuté la méthode `Application.launch()` – que l'on trouve dans le `main()` - l'application JavaFX s'initialise et appelle la méthode `start()` pour démarrer.

Plus de détails : <https://docs.oracle.com/javase/8/javafx/api/javafx/application/Application.html>

```
public class Main extends Application {

    public static void main (String [] args) {
        launch (args) ;
    }

    @Override
    public void start (Stage primaryStage) throws Exception {
        // primaryStage correspond a la fenetre principale de
        // l'application. Nous devons lui attribuer une scene
        // et pour cela nous devons prealablement creer l'arborescence
        // de noeuds graphiques constituant cette scene :
        Pane root = new PatientPane();
        // ou Pane est la classe générique des containers
        // ou PatientPane() est decrit plus bas

        // association de la scène contenant le container principal
        // en tant que scene de l'application
        Scene scenegraph = new Scene (root) ;
        primaryStage.setScene (scenegraph) ;
        primaryStage.setTitle ("Patient");
        primaryStage.show ();
    }
}

public class PatientPane extends BorderPane // ou HBox | VBox | GridPane | ....
{

    public PatientPane () {
        // code ajoutant le contenu du panneau Patient
    }
}
```

Scenegraph

JavaFX construit un graphe de scene (objet Scene) avec l'assemblage de composants que vous décrivez dans votre programme. La Scene contient la racine (root) de ce graphe de scene, autrement dit c'est le point d'entrée pour tous les contenus graphiques de l'application. Les noms des classes (Stage, Scene, ...) de JavaFX ont été choisis par les développeurs de JavaFX en référence au théâtre.

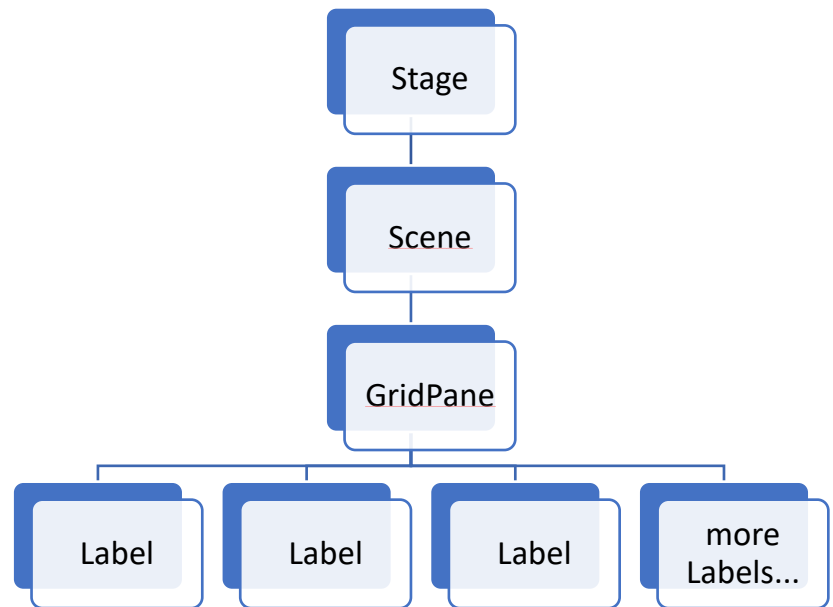
Ci-dessous l'exemple du Stage et de la Scene du TP1 :

La Scene est composée d'objets graphiques (des widgets et des containers) qui héritent tous de la classe Node.

Tous les objets graphiques du scenegraph ont donc des propriétés communes comme par exemple les transformations géométriques (scale, transform, translation).

Les Nodes les plus courants sont les widgets (du package `javafx.scene.control`), des composants graphiques interactifs que vous avez utilisés dans les trois premiers TP (User Interface Controls ou UI Controls en anglais).

Mais les Nodes peuvent aussi être de simples formes géométriques. JavaFX est très flexible.

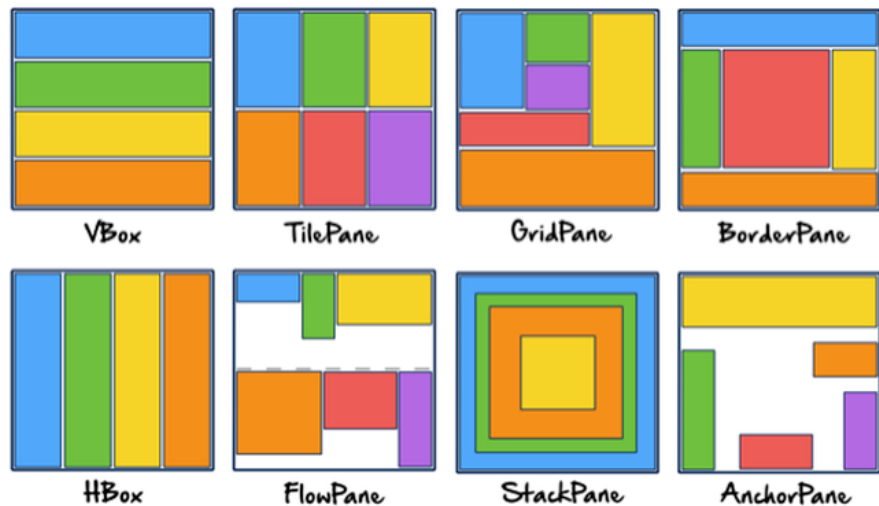


Plus de details : <https://docs.oracle.com/javase/8/javafx/api/javafx/scene/Node.html>

Layout

Un Layout (container ou gestionnaire de mise en page) est un container graphique qui hérite de la classe `javafx.scene.layout.Pane`. Il permet d'afficher un groupe de widgets (ou d'autres containers) suivant une disposition qui lui est propre. En imbriquant les différents types de conteneurs entre eux, on structure l'affichage afin d'aboutir à des interfaces complexes, comme on a pu le voir au TP3 avec le `TabPane` qui contenait un `Tab` qui contenait un `BorderPane` qui contenait lui-même plusieurs containers.

Les différents Layout permettent les dispositions suivantes :



Ces containers s'utilisent toujours de la même façon :

- 1) On crée le container :

```
HBox hbox = new HBox();
```
- 2) On ajoute d'autres widgets et containers à ce dernier :

```
hbox.getChildren().add( /* les composants que l'on veut mettre dedans */);
```
- 3) Puis on ajoute ce container lui-même à un autre, ou directement à la Scene.

Le BorderPane s'utilise avec setLeft/setRight/setTop/setCenter/setBottom.

Le GridPane s'utilise avec add/addColumn/addRow.

Widgets

Parmi les différents Node que l'on peut placer dans le scenegraph Scene, les widgets sont les plus sophistiqués, ils intègrent des comportements évolués. De part sa nature flexible et ses performances graphiques, JavaFX possède beaucoup d'extensions (des bibliothèques supplémentaires de widgets). Cependant pour nos TP, ceux proposés à la base seront :



Extrait de https://docs.oracle.com/javafx/2/ui_controls/overview.htm

La création de widgets se fait toujours de la même façon :

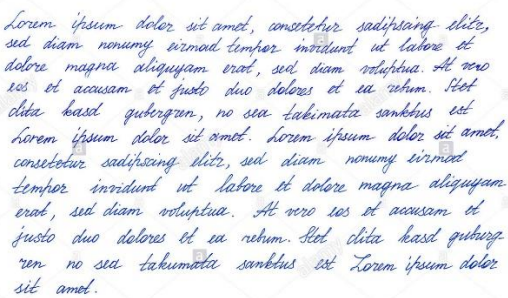
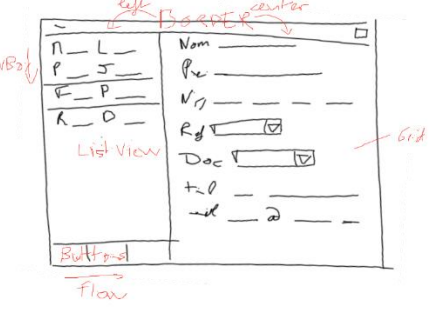
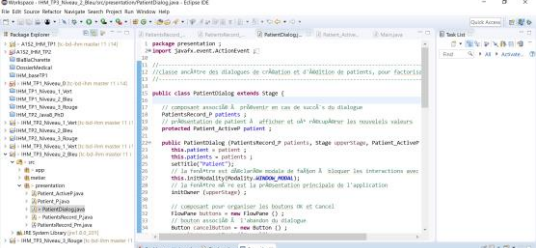
- 1) On crée le widget :
`Button newPatientButton = new Button();`
- 2) On paramètre le widget :
`newPatientButton.setText("Créer un nouveau patient");`
- 3) On l'ajoute à un container du scenegraph :
`myHBox.getChildren().add(newPatientButton);`
(sinon il n'apparaîtra jamais à l'écran !)

C'est tout ?

Une fenêtre et un scenegraph qui contient des objets graphiques, c'est quasiment tout pour faire une IHM. Il manque deux éléments :

- Que l'application fasse ce que le développeur (vous) a décidé quand l'utilisateur déclenche un événement (clic sur un bouton, tapez au clavier, etc) – c'est ce que nous voyons ci-dessous pour le TP4.
- Que l'application ressemble à ce que l'on veut afin que le rendu soit clair et lisible. C'est ce que nous verrons pour le TP5.

II. En résumé : comment on fait une IHM ?

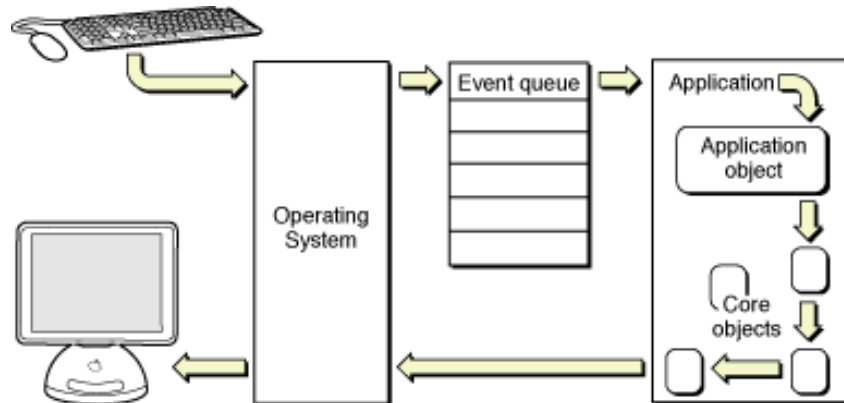
Etape 1		On part de la description d'une situation connue (par exemple la gestion des données patients dans une cabinet médical) pour des utilisateurs connus (secrétaires).
Etape 2		On dessine un prototype papier de l'interface (on en dessine souvent plusieurs pour explorer plusieurs choix et raffiner les détails au fur et à mesure).
Etape 3		On découpe l'interface en une série de containers (Layout) possibles, et on choisit les widgets. Il y a toujours plusieurs solutions.
Etape 4		On implémente le tout dans un nouveau projet JavaFX en créant une nouvelle classe pour chaque composant important.
Etape 5		On teste pour voir si tout ce qu'on avait prévu fonctionne bien ! On revient à l'étape 3 ou 4 pour améliorer la qualité du code et de l'interface elle-même.

Au fur et à mesure que l'on progresse, on peut utiliser de plus en plus de composants (TP3), leur ajouter des événements (TP4), connaître leur paramétrage pour améliorer l'ergonomie (TP5) et rendre le code plus structuré et plus lisible avec quelques règles de bon sens et un peu d'architecture logicielle (mini-projet).

III. Les événements

La programmation événementielle est un paradigme de programmation fondé sur les événements. Elle s'oppose à la programmation séquentielle. Le programme sera principalement défini par ses réactions aux différents événements qui peuvent se produire, par exemple un clic sur un bouton, un mouvement de souris ou la frappe d'une touche du clavier.

Autrement dit avec JavaFX le programme attend les actions (input events) de l'utilisateur.



Un événement ?

- Une petite structure de données (la classe `javafx.event.Event`) qui contient le moment du déclenchement de l'événement, son type, sa source (souris, clavier, ...), sa destination (target), et des données qui en dépendent comme la position (x,y) de la souris au moment d'un clic.
- La cible (target) d'un événement est l'élément (Node) sur lequel l'événement s'est produit (un bouton par exemple)
- C'est JavaFX qui reçoit les événements et les fait passer au composant en focus grâce à un gestionnaire d'événements.

Event Handler

- Vous devez ajouter un gestionnaire d'événements à aux composants que vous créez afin de définir ce que vous voulez qu'ils réalisent lorsque l'événement est déclenché par l'utilisateur.
- Si on a un `Button` `newPatientButton`, on procède en deux temps pour lui associer un comportement.

1) Création d'un gestionnaire d'événement

Gestionnaire d'événement avec une classe interne anonyme :

```
EventHandler handler = new EventHandler<ActionEvent>() {
    public void handle(ActionEvent event) {
        // on a cliqué sur le bouton
        System.out.println("Handling event " + event.getEventType());
        // on signifie à JavaFX que rien d'autre n'a besoin de cet événement
        event.consume();
    }
};
```

- Remarquer qu'on instancie une classe implémentant l'interface EventHandler en la définissant à la volée ! Ce mécanisme bien pratique pour des petites classes à usage unique s'appelle une [classe interne anonyme](#) – anonyme car on ne lui donne pas de nom.
- Noter qu'EventHandler est une interface Java.
- On peut très bien également créer une classe 'comme d'habitude' dans un fichier java séparé ou dans la même classe Java :

Gestionnaire d'événement avec une classe interne :

```
class MonGestionnairePourBoutonPatient implements EventHandler<ActionEvent> {
    @Override
    public void handle (ActionEvent event) {
        System.out.println("Handling event " + event.getEventType());
        event.consume();
    }
}
```

Vous pouvez choisir l'une ou l'autre approche, classe interne anonyme ou pas – l'une des deux écritures est plus compacte mais le résultat est strictement le même.

- 2) on l'associe au bouton en question :

```
newPatientButton.setOnAction (handler);
```

- et ça y est ! Votre bouton déclenche quelque chose !
- setOnAction() est une méthode pratique pour associer un gestionnaire d'événements ActionListener à un bouton – le cas le plus fréquent, mais n'importe quel gestionnaire d'événements peut être ajouté à un Node (le type de base des containers et des widgets associés à la Scene) grâce à la méthode **addEventHandler** :

```
newPatientButton.addEventHandler(MouseEvent.MOUSE_CLICKED, clicHandler);
```

[On trouve ici tous les méthodes pratiques](#) que JavaFX met en place pour simplifier l'association de gestionnaires d'événements.

En résumé

- Pour créer des comportements associés aux actions de l'utilisateur dans votre application, il faut créer des gestionnaires d'événements (EventHandler)
- Il ne faut pas oublier, une fois créés, de les associer avec le widget qui déclenche ce comportement !
- Voici un exemple de quelques types d'événements :

TP3 - IHM

Patients

Nichol John	Social Security Number : 123435665467899
Nicholson Jack	Gender : Male
	Date of birth : 1/1/1970
	Last Name : Nicholson
	First Name : Jack
	Attachement : John Nichol
	Referent Doctor : Henri-Désiré Landru

New Patient... Edit Patient... Delete

- Ajout du bouton SUPPRIMER

- Dans la classe `PatientsRecord_P`, ajouter un nouveau container (par exemple de type `HBox` ou `FlowPane` qu'on placera en bas du `BorderPane` à l'aide de la méthode `setBottom`
- Dans ce container on pourra ajouter un `Button` destiné à supprimer le patient courant, et on lui associera un `EventHandler`. Si vous effectuez cette suppression dans une méthode `deletePatient` définie par ailleurs dans votre classe, le code peut ressembler à ça :

```
Button deletePatientButton = new Button () ;
deletePatientButton.setText ("Delete") ;
deletePatientButton.setOnAction (new EventHandler<ActionEvent> () {
    @Override
    public void handle (ActionEvent event) {
        deletePatient (currentPatient) ;
    }
}) ;
```

- Création d'une fenêtre de dialogue de création de patients

- Cette fenêtre de dialogue devra contenir un composant de présentation `Patient_ActiveP` (tel que défini au TP 2) ainsi que deux `Button` permettant de valider la création ou de l'abandonner.
 - À vous de choisir les bons containers pour organiser l'agencement de ces composants...
- Créer pour cela une nouvelle classe `PatientDialog` qui héritera de la classe `Stage`

```
public class PatientDialog extends Stage { ... }
```

- De façon à s'afficher, on retrouvera à la fin du constructeur de cette classe des instructions qui seront similaires à celles de la fin de la méthode `start` de la classe `Main` (puisque'il s'agit aussi ici d'une fenêtre `Stage`), en supposant que dans cette classe votre container de plus haut niveau ait été nommé `globalPane` :

```
Scene scene = new Scene (globalPane) ;
this.setScene (scene) ;
this.setTitle ("Edit Patient") ;
this.show () ;
```

- On indique que notre nouvelle fenêtre est dépendante de la fenêtre principale :

```
this.initOwner (upperStage) ;
```

où `upperStage` est de type `Stage` et est passé en paramètre au constructeur de `PatientDialog` (il s'agit de la `primaryStage` de la classe `Main`).

- Notre nouvelle fenêtre doit être modale, autrement dit on veut bloquer les interactions avec la fenêtre principale et le reste de l'application le temps de son utilisation :

```
this.initModality(Modality.WINDOW_MODAL);
```

- l'action à associer au bouton d'annulation de création peut être très simple, du type :

```
this.close () ;
```

- l'action à associer au bouton de validation devra en plus :
 - communiquer avec le composant `Patient_ActiveP` pour récupérer le patient avec toutes les caractéristiques indiquées par l'utilisateur
 - transmettre ce patient à l'instance de la classe `PatientsRecord_P` qui avait été passée au constructeur pour que celle-ci puisse ajouter le nouveau patient dans sa liste de patients et ainsi le visualiser dans sa liste de patients
- Ajout du bouton NOUVEAU
 - Ce `Button` pourra être ajouté de la même façon que le précédent, et on lui associera également un `EventHandler` permettant cette fois d'invoquer une action permettant d'ajouter un patient dans la liste
 - Cette action consistera à faire apparaître une fenêtre de dialogue de création de patient.
- Ajout de la fonctionnalité de modification d'un patient :
 - Cela pourra se faire de façon similaire à ce que vous aurez fait pour la création de patient, par exemple :
 - vous pourrez envoyer au `PatientDialog` la présentation du patient courant (au lieu d'envoyer la présentation d'un nouveau patient "vide")
 - il suffira alors de vérifier, dans le code appelé en retour par le `PatientDialog`, si le patient renvoyé est déjà ou pas dans la liste des patients : s'il n'y est pas il faudra le rajouter, s'il y est il conviendra seulement de mettre à jour sa présentation en cours d'affichage dans la zone centrale du `PatientRecords_P`